

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5

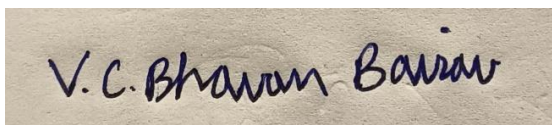
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I V.C.BHAVAN BAIRAV - 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1) For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

AIM:

To identify the functional dependencies and candidate key of the given relation.

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE STUDENT_COURSE (
  ->   SID INT,
  ->   SName VARCHAR(30),
  ->   CID VARCHAR(10),
  ->   CName VARCHAR(30),
  ->   Instructor VARCHAR(30),
  ->   Grade VARCHAR(5),
  ->   PRIMARY KEY (SID, CID)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO STUDENT_COURSE VALUES
  -> (101, 'Arun', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (101, 'Arun', 'C102', 'DSA', 'Dr.Meena', 'B'),
  -> (102, 'Meena', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (103, 'Kiran', 'C103', 'Networks', 'Dr.Suresh', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE;
+----+-----+-----+-----+-----+-----+
| SID | SName | CID  | CName  | Instructor | Grade |
+----+-----+-----+-----+-----+-----+
| 101 | Arun  | C101 | DBMS   | Dr.Ravi   | A     |
| 101 | Arun  | C102 | DSA    | Dr.Meena  | B     |
| 102 | Meena | C101 | DBMS   | Dr.Ravi   | A     |
| 103 | Kiran | C103 | Networks | Dr.Suresh | B     |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

QUERY 1 – Verify SID → SName

```
mysql> SELECT SID, COUNT(DISTINCT SName) AS Name_Count
-> FROM STUDENT_COURSE
-> GROUP BY SID;
+-----+-----+
| SID | Name_Count |
+-----+-----+
| 101 |          1 |
| 102 |          1 |
| 103 |          1 |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

```
mysql> SELECT CID,
-> COUNT(DISTINCT CName) AS Course_Count,
-> COUNT(DISTINCT Instructor) AS Instructor_Count
-> FROM STUDENT_COURSE
-> GROUP BY CID;
+-----+-----+-----+
| CID | Course_Count | Instructor_Count |
+-----+-----+-----+
| C101 |          1 |          1 |
| C102 |          1 |          1 |
| C103 |          1 |          1 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> █
```

Therefore:

- $CID \rightarrow CName$
- $CID \rightarrow Instructor$

QUERY 3 – Verify Candidate Key (SID, CID)

```
mysql> SELECT SID, CID, COUNT(*) AS Record_Count
-> FROM STUDENT_COURSE
-> GROUP BY SID, CID;
```

SID	CID	Record_Count
101	C101	1
101	C102	1
102	C101	1
103	C103	1

```
4 rows in set (0.00 sec)

mysql>
```

Every (SID, CID) combination uniquely identifies one record.

Therefore:

- $(SID, CID) \rightarrow Grade$
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- $SID \rightarrow SName$
- $CID \rightarrow CName$
- $CID \rightarrow Instructor$
- $(SID, CID) \rightarrow Grade$

CANDIDATE KEY:

- (SID, CID)

RESULT:

Thus, the functional dependencies were identified and **(SID, CID)** was determined as the candidate key of the STUDENT_COURSE relation successfully.

2) Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

AIM:

To demonstrate the process of converting an unnormalized relation into First Normal Form (1NF) by removing repeating and multi-valued attributes.

UNNORMALIZED RELATION:

Consider a RESEARCHER_PROJECT relation where a researcher can work on multiple projects.

```
mysql> CREATE TABLE RESEARCHER_PROJECT_UNF (
  ->     ResearcherID VARCHAR(10),
  ->     ResearcherName VARCHAR(30),
  ->     Projects VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCHER_PROJECT_UNF VALUES
  -> ('R101', 'Arun', 'AI, Robotics'),
  -> ('R102', 'Meena', 'IoT'),
  -> ('R103', 'Kiran', 'Cyber Security, Cloud');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCHER_PROJECT_UNF;
+-----+-----+-----+
| ResearcherID | ResearcherName | Projects |
+-----+-----+-----+
| R101         | Arun           | AI, Robotics |
| R102         | Meena          | IoT |
| R103         | Kiran          | Cyber Security, Cloud |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

PROBLEM:

- Projects = AI, Robotics
- Projects = Cyber Security, Cloud

These are non-atomic/multiple values, violating 1NF.

CONVERT TO 1NF:

Each project is placed in a **separate row**.

```
mysql> CREATE TABLE RESEARCHER_PROJECT_1NF (  
->     ResearcherID VARCHAR(10),  
->     ResearcherName VARCHAR(30),  
->     Project VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO RESEARCHER_PROJECT_1NF VALUES  
-> ('R101', 'Arun', 'AI'),  
-> ('R101', 'Arun', 'Robotics'),  
-> ('R102', 'Meena', 'IoT'),  
-> ('R103', 'Kiran', 'Cyber Security'),  
-> ('R103', 'Kiran', 'Cloud');  
Query OK, 5 rows affected (0.01 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM RESEARCHER_PROJECT_1NF;  
+-----+-----+-----+  
| ResearcherID | ResearcherName | Project      |  
+-----+-----+-----+  
| R101         | Arun           | AI           |  
| R101         | Arun           | Robotics     |  
| R102         | Meena          | IoT          |  
| R103         | Kiran          | Cyber Security |  
| R103         | Kiran          | Cloud        |  
+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> _
```

STEP-BY-STEP JUSTIFICATION:

Before 1NF:

- $R101 \rightarrow \text{AI, Robotics}$
- $R102 \rightarrow \text{IoT}$
- $R103 \rightarrow \text{Cyber Security, Cloud}$

After 1NF:

- $R101 \rightarrow \text{AI}$
- $R101 \rightarrow \text{Robotics}$
- $R102 \rightarrow \text{IoT}$
- $R103 \rightarrow \text{Cyber Security}$
- $R103 \rightarrow \text{Cloud}$

Now every attribute contains only one atomic value.

RESULT:

Thus, the unnormalized RESEARCHER_PROJECT_UNF relation was successfully converted into 1NF by removing the repeating/multi-valued Projects attribute and storing each project as a separate row.

3) Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

AIM:

To analyze the given relation using functional dependencies and determine whether decomposition is required to achieve Second Normal Form (2NF).

GIVEN RELATION:

```
mysql> CREATE TABLE RESEARCH_RECORD (  
  ->   A INT PRIMARY KEY,  
  ->   B VARCHAR(20),  
  ->   C VARCHAR(20),  
  ->   D VARCHAR(20),  
  ->   E VARCHAR(20)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO RESEARCH_RECORD VALUES  
  -> (101, 'B1', 'C1', 'D1', 'E1'),  
  -> (102, 'B2', 'C2', 'D2', 'E2'),  
  -> (103, 'B3', 'C3', 'D3', 'E3'),  
  -> (104, 'B4', 'C4', 'D4', 'E4');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM RESEARCH_RECORD;  
+-----+-----+-----+-----+-----+  
| A     | B     | C     | D     | E     |  
+-----+-----+-----+-----+-----+  
| 101   | B1    | C1    | D1    | E1    |  
| 102   | B2    | C2    | D2    | E2    |  
| 103   | B3    | C3    | D3    | E3    |  
| 104   | B4    | C4    | D4    | E4    |  
+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

QUERY – Verify Candidate Key A

```
mysql> CREATE TABLE RESEARCH_RECORD (
  ->   A INT PRIMARY KEY,
  ->   B VARCHAR(20),
  ->   C VARCHAR(20),
  ->   D VARCHAR(20),
  ->   E VARCHAR(20)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_RECORD VALUES
  -> (101, 'B1', 'C1', 'D1', 'E1'),
  -> (102, 'B2', 'C2', 'D2', 'E2'),
  -> (103, 'B3', 'C3', 'D3', 'E3'),
  -> (104, 'B4', 'C4', 'D4', 'E4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT A,
  ->   COUNT(DISTINCT B) AS B_count,
  ->   COUNT(DISTINCT C) AS C_count,
  ->   COUNT(DISTINCT D) AS D_count,
  ->   COUNT(DISTINCT E) AS E_count
  -> FROM RESEARCH_RECORD
  -> GROUP BY A;
+-----+-----+-----+-----+
| A    | B_count | C_count | D_count | E_count |
+-----+-----+-----+-----+
| 101  | 1       | 1       | 1       | 1       |
| 102  | 1       | 1       | 1       | 1       |
| 103  | 1       | 1       | 1       | 1       |
| 104  | 1       | 1       | 1       | 1       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> _
```

$A^+ = \{A, B, C, D, E\}$

Therefore, Candidate Key = A.

CHECK FOR PARTIAL DEPENDENCY

A partial dependency occurs when a non-key attribute depends on part of a composite candidate key.

Here:

- Candidate Key = A

The key contains only one attribute.

Therefore, a partial dependency cannot exist. Hence the table is already in 2NF therefore No decomposition required

RESULT:

Thus, the given relation $R(A,B,C,D,E)$ is already in 2NF, since its candidate key is the single attribute A and therefore no partial dependency exists. No decomposition is required for 2NF.

4) Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

AIM:

To apply Third Normal Form (3NF) decomposition to the given relation by removing transitive dependencies.

GIVEN RELATION:

```
mysql> CREATE TABLE EMPLOYEE (
->     EmpID INT PRIMARY KEY,
->     EmpName VARCHAR(30),
->     DeptID VARCHAR(10),
->     DeptName VARCHAR(30),
->     ManagerID VARCHAR(10)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE VALUES
-> (101, 'Arun', 'D01', 'Research', 'M201'),
-> (102, 'Meena', 'D01', 'Research', 'M201'),
-> (103, 'Kiran', 'D02', 'Robotics', 'M202'),
-> (104, 'Ravi', 'D03', 'AI Lab', 'M203');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Arun    | D01    | Research | M201      |
| 102   | Meena   | D01    | Research | M201      |
| 103   | Kiran   | D02    | Robotics | M202      |
| 104   | Ravi    | D03    | AI Lab  | M203      |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Here, the transitive dependency is:

- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

DeptName and ManagerID depend on EmpID **through DeptID**, creating a transitive dependency.

DECOMPOSE INTO 3NF

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE EMPLOYEE_3NF (  
  ->     EmpID INT PRIMARY KEY,  
  ->     EmpName VARCHAR(30),  
  ->     DeptID VARCHAR(10)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE DEPARTMENT (  
  ->     DeptID VARCHAR(10) PRIMARY KEY,  
  ->     DeptName VARCHAR(30),  
  ->     ManagerID VARCHAR(10)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO EMPLOYEE_3NF VALUES  
  -> (101, 'Arun', 'D01'),  
  -> (102, 'Meena', 'D01'),  
  -> (103, 'Kiran', 'D02'),  
  -> (104, 'Ravi', 'D03');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO DEPARTMENT VALUES  
  -> ('D01', 'Research', 'M201'),  
  -> ('D02', 'Robotics', 'M202'),  
  -> ('D03', 'AI Lab', 'M203');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM EMPLOYEE_3NF;  
+-----+-----+-----+  
| EmpID | EmpName | DeptID |  
+-----+-----+-----+  
| 101   | Arun    | D01    |  
| 102   | Meena   | D01    |  
| 103   | Kiran   | D02    |  
| 104   | Ravi    | D03    |  
+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> █
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName | ManagerID |
+-----+-----+-----+
| D01    | Research | M201      |
| D02    | Robotics | M202      |
| D03    | AI Lab   | M203      |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

RESULT:

Thus, the EMPLOYEE relation was successfully decomposed into 3NF by removing the transitive dependency and separating department details into the DEPARTMENT relation.

5) Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

AIM:

To verify whether the given relation is in Boyce-Codd Normal Form (BCNF) and decompose it into BCNF if required.

GIVEN RELATION:


```
MySQL 8.0 Command Line Client
Database changed
mysql> CREATE TABLE RELATION_ABCD (
  ->   A VARCHAR(10),
  ->   B VARCHAR(10),
  ->   C VARCHAR(10),
  ->   D VARCHAR(10),
  ->   PRIMARY KEY (A, B)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO RELATION_ABCD VALUES
  -> ('A1','B1','C1','D1'),
  -> ('A2','B2','C2','D2'),
  -> ('A3','B3','C3','D3'),
  -> ('A4','B4','C4','D4');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RELATION_ABCD;
+----+-----+-----+-----+
| A  | B    | C    | D    |
+----+-----+-----+-----+
| A1 | B1   | C1   | D1   |
| A2 | B2   | C2   | D2   |
| A3 | B3   | C3   | D3   |
| A4 | B4   | C4   | D4   |
+----+-----+-----+-----+
4 rows in set (0.01 sec)
```

FUNCTIONAL DEPENDENCIES:

- $AB \rightarrow C$
- $C \rightarrow D$
- $D \rightarrow A$

VERIFY CANDIDATE KEY AB

```
mysql> SELECT A, B, COUNT(*) AS Record_Count
-> FROM RELATION_ABCD
-> GROUP BY A, B;
+-----+-----+-----+
| A | B | Record_Count |
+-----+-----+-----+
| A1 | B1 | 1 |
| A2 | B2 | 1 |
| A3 | B3 | 1 |
| A4 | B4 | 1 |
+-----+-----+-----+
4 rows in set (0.01 sec)

mysql> █
```

Therefore, AB is a candidate key.

CREATE FIRST BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CD (
-> C VARCHAR(10) PRIMARY KEY,
-> D VARCHAR(10)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RELATION_CD VALUES
-> ('C1','D1'),
-> ('C2','D2'),
-> ('C3','D3'),
-> ('C4','D4');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM RELATION_CD;
+-----+-----+
| C | D |
+-----+-----+
| C1 | D1 |
| C2 | D2 |
| C3 | D3 |
| C4 | D4 |
+-----+-----+
4 rows in set (0.00 sec)
```

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE RELATION_ABC (  
->   A VARCHAR(10),  
->   B VARCHAR(10),  
->   C VARCHAR(10),  
->   PRIMARY KEY (A, B)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> INSERT INTO RELATION_ABC VALUES  
-> ('A1','B1','C1'),  
-> ('A2','B2','C2'),  
-> ('A3','B3','C3'),  
-> ('A4','B4','C4');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM RELATION_ABC;  
+----+-----+-----+  
| A  | B  | C  |  
+----+-----+-----+  
| A1 | B1 | C1 |  
| A2 | B2 | C2 |  
| A3 | B3 | C3 |  
| A4 | B4 | C4 |  
+----+-----+-----+  
4 rows in set (0.00 sec)
```

From:

- $C \rightarrow D$

Decompose into:

- $R_1(C,D)$
- $R_2(A,B,C)$

SECOND BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CA (  
-> C VARCHAR(10) PRIMARY KEY,  
-> A VARCHAR(10)  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> CREATE TABLE RELATION_BC (  
-> B VARCHAR(10),  
-> C VARCHAR(10),  
-> PRIMARY KEY (B, C)  
-> );
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> INSERT INTO RELATION_CA VALUES  
-> ('C1','A1'),  
-> ('C2','A2'),  
-> ('C3','A3'),  
-> ('C4','A4');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> INSERT INTO RELATION_BC VALUES  
-> ('B1','C1'),  
-> ('B2','C2'),  
-> ('B3','C3'),  
-> ('B4','C4');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM RELATION_CA;
```

C	A
C1	A1
C2	A2
C3	A3
C4	A4

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM RELATION_BC;
```

B	C
B1	C1
B2	C2
B3	C3
B4	C4

4 rows in set (0.00 sec)

Final Decomposition:

- R1(C,D)
- R2(C,A)
- R3(B,C)

RESULT:

The relation R(A,B,C,D) is not in BCNF. It is successfully decomposed into: (C,D), (C,A), and (B,C).

6) Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

AIM:

To experimentally verify whether a given decomposition is lossless using the tabular (Chase) algorithm.

GIVEN RELATION:

```
mysql> CREATE TABLE RESEARCH_PROJECT (
->   ResearcherID VARCHAR(10),
->   ProjectID VARCHAR(10),
->   ProjectName VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_PROJECT VALUES
-> ('R101','P101','AI Research'),
-> ('R102','P102','Robotics'),
-> ('R103','P103','Cyber Security'),
-> ('R104','P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCH_PROJECT;
+-----+-----+-----+
| ResearcherID | ProjectID | ProjectName |
+-----+-----+-----+
| R101         | P101      | AI Research |
| R102         | P102      | Robotics    |
| R103         | P103      | Cyber Security |
| R104         | P104      | Cloud Computing |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

CREATE DECOMPOSED RELATIONS

```
mysql> CREATE TABLE RESEARCHER_PROJECT (
->     ResearcherID VARCHAR(10),
->     ProjectID VARCHAR(10)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE PROJECT_DETAILS (
->     ProjectID VARCHAR(10),
->     ProjectName VARCHAR(30)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RESEARCHER_PROJECT VALUES
-> ('R101','P101'),
-> ('R102','P102'),
-> ('R103','P103'),
-> ('R104','P104');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PROJECT_DETAILS VALUES
-> ('P101','AI Research'),
-> ('P102','Robotics'),
-> ('P103','Cyber Security'),
-> ('P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

CHASE TABLE:

Initial Chase Table:

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	b3
R2	b1	a2	a3

FD: $\text{ProjectID} \rightarrow \text{ProjectName}$

Since both rows have the same ProjectID (a2),

ProjectName takes the common distinguished value (a3).

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	a3
R2	b1	a2	a3

A row contains all distinguished symbols.

Therefore, the decomposition is LOSSLESS.

VERIFY USING SQL JOIN:

```
mysql> SELECT r.ResearcherID,
->         r.ProjectID,
->         p.ProjectName
-> FROM RESEARCHER_PROJECT r
-> JOIN PROJECT_DETAILS p
-> ON r.ProjectID = p.ProjectID;
+-----+-----+-----+
| ResearcherID | ProjectID | ProjectName |
+-----+-----+-----+
| R101        | P101     | AI Research |
| R102        | P102     | Robotics    |
| R103        | P103     | Cyber Security |
| R104        | P104     | Cloud Computing |
+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

RESULT:

Thus, using the Chase algorithm, the decomposition

- R1(ResearcherID, ProjectID)
- R2(ProjectID, ProjectName)

was experimentally verified to be lossless-join, since the chase table produces a row containing all distinguished symbols and the join reconstructs the original relation.

7) Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

AIM:

To identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose the relation into Fourth Normal Form (4NF).

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE TEACHER (
->   TID VARCHAR(10),
->   Subject VARCHAR(30),
->   Hobby VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER VALUES
-> ('T01','DBMS','Cricket'),
-> ('T01','DBMS','Music'),
-> ('T01','DSA','Cricket'),
-> ('T01','DSA','Music'),
-> ('T02','Networks','Reading'),
-> ('T02','AI','Reading');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER;
+-----+-----+-----+
| TID | Subject | Hobby |
+-----+-----+-----+
| T01 | DBMS    | Cricket |
| T01 | DBMS    | Music   |
| T01 | DSA     | Cricket |
| T01 | DSA     | Music   |
| T02 | Networks | Reading |
| T02 | AI      | Reading |
+-----+-----+-----+
6 rows in set (0.00 sec)
```


MULTI-VALUED DEPENDENCIES

- $TID \twoheadrightarrow Subject$
- $TID \twoheadrightarrow Hobby$

DECOMPOSE INTO 4NF:

```
mysql> CREATE TABLE TEACHER_SUBJECT (  
-> TID VARCHAR(10),  
-> Subject VARCHAR(30),  
-> PRIMARY KEY (TID, Subject)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> INSERT INTO TEACHER_SUBJECT VALUES  
-> ('T01','DBMS'),  
-> ('T01','DSA'),  
-> ('T02','Networks'),  
-> ('T02','AI');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM TEACHER_SUBJECT;  
+-----+-----+  
| TID | Subject |  
+-----+-----+  
| T01 | DBMS    |  
| T01 | DSA     |  
| T02 | AI      |  
| T02 | Networks|  
+-----+-----+  
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE TEACHER_HOBBY (
  ->     TID VARCHAR(10),
  ->     Hobby VARCHAR(30),
  ->     PRIMARY KEY (TID, Hobby)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER_HOBBY VALUES
  -> ('T01','Cricket'),
  -> ('T01','Music'),
  -> ('T02','Reading');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+-----+
| TID | Hobby |
+-----+-----+
| T01 | Cricket |
| T01 | Music |
| T02 | Reading |
+-----+-----+
3 rows in set (0.00 sec)
```

- TID $\twoheadrightarrow$ Subject
- TID $\twoheadrightarrow$ Hobby

Therefore, the original relation contains independent multi-valued dependencies.

Decomposition:

- TEACHER_SUBJECT(TID, Subject)
- TEACHER_HOBBY(TID, Hobby)

RESULT:

Thus, the multi-valued dependencies were identified and the TEACHER relation was successfully decomposed into 4NF.

8) Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

AIM:

To demonstrate how join dependencies lead to redundancy and decompose a relation into Fifth Normal Form (5NF) or Project-Join Normal Form (PJNF).

GIVEN RELATION:

```
Database changed
mysql> CREATE TABLE SUPPLIER_PRODUCT_PORT (
  ->     Supplier VARCHAR(10),
  ->     Product VARCHAR(10),
  ->     Port VARCHAR(20)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SUPPLIER_PRODUCT_PORT VALUES
  -> ('S01','P01','Chennai'),
  -> ('S01','P01','Mumbai'),
  -> ('S01','P02','Chennai'),
  -> ('S01','P02','Mumbai'),
  -> ('S02','P01','Chennai'),
  -> ('S02','P01','Mumbai');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SUPPLIER_PRODUCT_PORT;
+-----+-----+-----+
| Supplier | Product | Port |
+-----+-----+-----+
| S01      | P01     | Chennai |
| S01      | P01     | Mumbai |
| S01      | P02     | Chennai |
| S01      | P02     | Mumbai |
| S02      | P01     | Chennai |
| S02      | P01     | Mumbai |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

REDUNDANCY

For S01, the supplier-product and supplier-port associations create multiple combinations:

- $S01 \rightarrow P01, P02$
- $S01 \rightarrow \text{Chennai, Mumbai}$

This produces repeated combinations in the original three-attribute relation.

DECOMPOSITION INTO 5NF

```
mysql> CREATE TABLE SUPPLIER_PRODUCT (  
->     Supplier VARCHAR(10),  
->     Product VARCHAR(10),  
->     PRIMARY KEY (Supplier, Product)  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO SUPPLIER_PRODUCT VALUES  
-> ('S01','P01'),  
-> ('S01','P02'),  
-> ('S02','P01');
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM SUPPLIER_PRODUCT;
```

```
+-----+-----+  
| Supplier | Product |  
+-----+-----+  
| S01      | P01     |  
| S01      | P02     |  
| S02      | P01     |  
+-----+-----+  
3 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE SUPPLIER_PORT (  
->     Supplier VARCHAR(10),  
->     Port VARCHAR(20),  
->     PRIMARY KEY (Supplier, Port)  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO SUPPLIER_PORT VALUES  
-> ('S01','Chennai'),  
-> ('S01','Mumbai'),  
-> ('S02','Chennai'),  
-> ('S02','Mumbai');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM SUPPLIER_PORT;
```

Supplier	Port
S01	Chennai
S01	Mumbai
S02	Chennai
S02	Mumbai

4 rows in set (0.00 sec)

```
mysql> CREATE TABLE PRODUCT_PORT (  
->     Product VARCHAR(10),  
->     Port VARCHAR(20),  
->     PRIMARY KEY (Product, Port)  
-> );
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> INSERT INTO PRODUCT_PORT VALUES  
-> ('P01','Chennai'),  
-> ('P01','Mumbai'),  
-> ('P02','Chennai'),  
-> ('P02','Mumbai');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM PRODUCT_PORT;
```

Product	Port
P01	Chennai
P01	Mumbai
P02	Chennai
P02	Mumbai

4 rows in set (0.00 sec)

Original Relation:

- SUPPLIER_PRODUCT_PORT(Supplier, Product, Port)

Decomposition:

- SUPPLIER_PRODUCT(Supplier, Product)
- SUPPLIER_PORT(Supplier, Port)
- PRODUCT_PORT(Product, Port)

The three relations remove the redundancy caused by the independent associations and represent the join dependency separately.

RESULT:

Thus, the join dependency and associated redundancy were demonstrated, and the relation was decomposed into 5NF (PJNF) relations successfully.

9) Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

AIM:

To identify insertion, deletion and update anomalies in a poorly designed college database and normalize the relation to Third Normal Form (3NF).

GIVEN RELATION:

```
mysql> CREATE TABLE COLLEGE_COURSE (
  ->     StudentID VARCHAR(10),
  ->     StudentName VARCHAR(30),
  ->     CourseID VARCHAR(10),
  ->     CourseName VARCHAR(30),
  ->     FacultyID VARCHAR(10),
  ->     FacultyName VARCHAR(30)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO COLLEGE_COURSE VALUES
  -> ('S101','Arun','C01','DBMS','F01','Dr.Ravi'),
  -> ('S102','Meena','C01','DBMS','F01','Dr.Ravi'),
  -> ('S103','Kiran','C02','Networks','F02','Dr.Suresh'),
  -> ('S104','Ravi','C03','AI','F03','Dr.Meena');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE_COURSE;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | CourseID | CourseName | FacultyID | FacultyName |
+-----+-----+-----+-----+-----+-----+
| S101      | Arun        | C01      | DBMS       | F01       | Dr.Ravi     |
| S102      | Meena       | C01      | DBMS       | F01       | Dr.Ravi     |
| S103      | Kiran       | C02      | Networks   | F02       | Dr.Suresh   |
| S104      | Ravi        | C03      | AI         | F03       | Dr.Meena    |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Functional Dependencies

- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$
- $\text{FacultyID} \rightarrow \text{FacultyName}$

Therefore, there is a transitive dependency:

$\text{CourseID} \rightarrow \text{FacultyID} \rightarrow \text{FacultyName}$

ANOMALIES:

1. Update Anomaly

If the faculty name Dr.Ravi changes, it must be updated in every row containing F01.

F01 → Dr.Ravi

Updating only one row causes inconsistent data.

2. Insertion Anomaly

A new course/faculty cannot be inserted unless a student is registered for that course.

3. Deletion Anomaly

If the last student registered for a course is deleted, the course and faculty information may also be lost.

DECOMPOSE INTO 3NF:

```
mysql> CREATE TABLE COLLEGE_STUDENT (  
->     StudentID VARCHAR(10) PRIMARY KEY,  
->     StudentName VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO COLLEGE_STUDENT VALUES  
-> ('S101','Arun'),  
-> ('S102','Meena'),  
-> ('S103','Kiran'),  
-> ('S104','Ravi');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM COLLEGE_STUDENT;  
+-----+-----+  
| StudentID | StudentName |  
+-----+-----+  
| S101      | Arun        |  
| S102      | Meena       |  
| S103      | Kiran       |  
| S104      | Ravi        |  
+-----+-----+  
4 rows in set (0.00 sec)
```



```
mysql> CREATE TABLE COLLEGE_FACULTY (
->     FacultyID VARCHAR(10) PRIMARY KEY,
->     FacultyName VARCHAR(30)
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO COLLEGE_FACULTY VALUES
-> ('F01','Dr.Ravi'),
-> ('F02','Dr.Suresh'),
-> ('F03','Dr.Meena');
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM COLLEGE_FACULTY;
```

FacultyID	FacultyName
F01	Dr.Ravi
F02	Dr.Suresh
F03	Dr.Meena

3 rows in set (0.00 sec)

```
mysql> CREATE TABLE COLLEGE_COURSE_3NF (
->     CourseID VARCHAR(10) PRIMARY KEY,
->     CourseName VARCHAR(30),
->     FacultyID VARCHAR(10),
->     FOREIGN KEY (FacultyID)
->     REFERENCES COLLEGE_FACULTY(FacultyID)
-> );
```

Query OK, 0 rows affected (0.08 sec)

```
mysql> INSERT INTO COLLEGE_COURSE_3NF VALUES
-> ('C01','DBMS','F01'),
-> ('C02','Networks','F02'),
-> ('C03','AI','F03');
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM COLLEGE_COURSE_3NF;
```

CourseID	CourseName	FacultyID
C01	DBMS	F01
C02	Networks	F02
C03	AI	F03

3 rows in set (0.00 sec)

Original:

COLLEGE_COURSE

- (StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

3NF Decomposition:

- COLLEGE_STUDENT(StudentID, StudentName)
- COLLEGE_COURSE_3NF(CourseID, CourseName, FacultyID)
- COLLEGE_FACULTY(FacultyID, FacultyName)

RESULT:

Thus, the insertion, deletion and update anomalies were identified, and the poorly designed college database was successfully normalized into 3NF by removing the transitive dependency.

10) Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

AIM:

To compute the closure of attribute sets for the given functional dependencies and determine the candidate key of the relation.

GIVEN RELATION:

```
mysql> CREATE TABLE RESEARCH (
  ->   ResearcherID VARCHAR(10),
  ->   ProjectID VARCHAR(10),
  ->   ProjectName VARCHAR(30),
  ->   LabID VARCHAR(10),
  ->   LabName VARCHAR(30),
  ->   PRIMARY KEY (ResearcherID)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RESEARCH VALUES
  -> ('R101','P101','AI Research','L01','AI Lab'),
  -> ('R102','P102','Robotics','L02','Robotics Lab'),
  -> ('R103','P103','Cyber Security','L03','Security Lab'),
  -> ('R104','P104','Cloud Computing','L04','Cloud Lab');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCH;
```

ResearcherID	ProjectID	ProjectName	LabID	LabName
R101	P101	AI Research	L01	AI Lab
R102	P102	Robotics	L02	Robotics Lab
R103	P103	Cyber Security	L03	Security Lab
R104	P104	Cloud Computing	L04	Cloud Lab

```
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- ResearcherID $\rightarrow$ ProjectID
- ProjectID $\rightarrow$ ProjectName, LabID
- LabID $\rightarrow$ LabName

ATTRIBUTE CLOSURE

CLOSURE OF RESEARCHERID

ResearcherID $\rightarrow$ {ResearcherID}

ResearcherID $\rightarrow$ ProjectID = {ResearcherID, ProjectID}

ProjectID $\rightarrow$ ProjectName, LabID = {ResearcherID, ProjectID, ProjectName, LabID}

LabID $\rightarrow$ LabName = {ResearcherID, ProjectID, ProjectName, LabID, LabName}

Therefore:

- ResearcherID $\rightarrow$ {ResearcherID, ProjectID, ProjectName, LabID, LabName}

Since the closure contains **all attributes**, ResearcherID is a **superkey**.

Since no smaller subset of ResearcherID exists:

- Candidate Key = ResearcherID

SQL VERIFICATION:

```
mysql> SELECT ResearcherID,
-> COUNT(DISTINCT ProjectID) AS Project_Count,
-> COUNT(DISTINCT ProjectName) AS ProjectName_Count,
-> COUNT(DISTINCT LabID) AS Lab_Count,
-> COUNT(DISTINCT LabName) AS LabName_Count
-> FROM RESEARCH
-> GROUP BY ResearcherID;
```

ResearcherID	Project_Count	ProjectName_Count	Lab_Count	LabName_Count
R101	1	1	1	1
R102	1	1	1	1
R103	1	1	1	1
R104	1	1	1	1

4 rows in set (0.00 sec)

RESULT:

Thus, the closure of ResearcherID was computed successfully, and ResearcherID was determined to be the candidate key of the RESEARCH relation.